VIT®
BHOPAL
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSA4008 - APPLIED MACHINE LEARNING

REG.NO : 23BAI10917

NAME : Vijay Rajesh R

BRANCH : BTECH - AIML

SEMESTER: Fall Semester 2025-26

INDEX

EXP.NO: 01	IMPLEMENT DECISION TREE LEARNING
DATE: 09.07.25	

AIM

To build and evaluate a Decision Tree Regression model on the Insurance dataset to predict medical insurance charges.

PROCEDURE

1. Import the necessary libraries:
   Use pandas, numpy for data handling; matplotlib and seaborn for visualization; and sklearn for machine learning functions.
2. Load the dataset:
   Load the insurance.csv file into a pandas DataFrame using pd.read_csv().
3. Preprocess the data:
   - Check for any missing values using isnull().sum().
   - Convert categorical columns (like sex, smoker, and region) into numerical form using encoding methods.
4. Split the features and target:
   - Define independent features (e.g., age, sex, bmi, etc.).
   - Set the dependent variable (charges) as the target.
5. Divide the data into training and testing sets:
   Use train_test_split() to split the data (e.g., 80% for training, 20% for testing).
6. Train the model:
   Initialize and train a DecisionTreeRegressor on the training data.
7. Make predictions:
   Use the trained model to predict charges for the test set.
8. Evaluate the model:
   Calculate Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score using sklearn metrics.

PROGRAM

```
# 1. Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load the dataset
file = pd.read_csv("insurance.csv")

# 3. Check for missing values
print(file.isnull().sum())

# 4. Encode categorical variables
file['sex'] = file['sex'].map({'male': 0, 'female': 1})
file['smoker'] = file['smoker'].map({'no': 0, 'yes': 1})
file['region'] = file['region'].map({'southwest': 0,
'southeast': 1, 'northwest': 2, 'northeast': 3})

# 5. Split features and target
X = file.drop('charges', axis=1)
y = file['charges']

# 6. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# 7. Train the Decision Tree Regressor
regressor = DecisionTreeRegressor()
regressor.fit(X_train, y_train)

# 8. Make predictions
y_pred = regressor.predict(X_test)

# 9. Evaluate the model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R2 Score:", r2)
```

INPUT

Note : The dataset is insurance.csv which has more number of data values. So given below is an first few data from the entire dataset.

```
age,sex,bmi,children,smoker,region,charges
19,female,27.9,0,yes,southwest,16884.924
18,male,33.77,1,no,southeast,1725.5523
28,male,33,3,no,southeast,4449.462
33,male,22.705,0,no,northwest,21984.47061
```

OUTPUT

```
MSE :  47329009.57964182
RMSE :  6879.608243180844
R^2 :  0.7011766883576342
```

RESULT

The Decision Tree Regressor model yielded a Mean Squared Error (**MSE)** of **47,329,009.58**, a Root Mean Squared Error **(RMSE)** of **6,879.61**, and an **R^2** score of **0.70**. This indicates that the model explains around **70%** of the variance in the insurance charges, providing a moderately accurate prediction, though there is room for improvement through hyperparameter tuning or more advanced models.

EXP.NO: 02	IMPLEMENT LOGISTIC REGRESSION
DATE: 09.07.25	

AIM

To implement Logistic Regression using Python for binary classification and evaluate its performance on a dataset.

PROCEDURE

1. Import Required Libraries
   Import necessary Python libraries such as pandas, numpy, matplotlib, and scikit-learn.

2. Load the Dataset
   Read the dataset using pandas.read_csv() and display the top rows using .head().

3. Check for Missing Values
   Use .isnull().sum() to verify if the dataset has any missing data.

4. Data Preprocessing
   Convert categorical columns into numerical format using get_dummies() or LabelEncoder if required.

5. Split Features and Target
   Define X (features) and y (target variable).

6. Split Data into Train and Test Sets
   Use train_test_split() to divide the dataset (e.g., 80% training and 20% testing).

7. Import and Train the Logistic Regression Model
   Create an instance of LogisticRegression and fit it to the training data using .fit().

8. Make Predictions
   Predict the target on the test set using .predict().

9. Evaluate the Model
   Evaluate model performance using metrics such as accuracy_score, confusion_matrix, and classification_report.

10. Display Results
    Print all evaluation metrics and visualize confusion matrix (optional).

PROGRAM

```
#1-----------------------------------
import pandas as pd
import numpy as np

file = pd.read_csv("insurance.csv")
file.head(3)

#2-----------------------------------
file.isnull().sum()
file.head(3)

#3-----------------------------------
n_file = pd.get_dummies(file, drop_first=True)
n_file.head(3)

#4-----------------------------------
x = n_file.drop('charges', axis=1)
y = n_file['charges']

#5-----------------------------------
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2)

#6-----------------------------------
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x_train, y_train)

#7-----------------------------------
y_pred = model.predict(x_test)

#8-----------------------------------
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MSE : ", mse)
print("RMSE : ", rmse)
print("R2 Score : ", r2)
```

INPUT

```
age,sex,bmi,children,smoker,region,charges
52,female,44.7,3,no,southwest,11411.685
50,male,30.97,3,no,northwest,10600.5483
18,female,31.92,0,no,northeast,2205.9808
18,female,36.85,0,no,southeast,1629.8335
21,female,25.8,0,no,southwest,2007.945
61,female,29.07,0,yes,northwest,29141.3603
```

OUTPUT

```
MSE :  30218435.943220887
RMSE :  5497.1297913748485
R^2 :  0.8097890765450464
```

RESULT

The Linear Regression Model achieved a Mean Squared Error **(MSE)** of approximately **30,218,435.94**, a Root Mean Squared Error **(RMSE)** of around **5497.13**, and an **R^2** score of **0.81**. This indicates that the model explains **81%** of the variance in the target variable, showing good predictive performance and a reasonably accurate fit for the dataset used.

EXP.NO: 03	IMPLEMENT CLASSIFICATION USING MULTILAYER PERCEPTRON
DATE: 13.07.25	

AIM

To implement a Multilayer Perceptron (MLPClassifier) to classify medical insurance charges into categories (Low, Medium, High) using the insurance.csv dataset and evaluate the model's performance using accuracy and classification metrics.

PROCEDURE

1. Import Required Libraries
   Import necessary Python libraries such as pandas, numpy, StandardScaler, LabelEncoder, train_test_split, MLPClassifier, and evaluation metrics from scikit-learn.
2. Load the Dataset
   Read the dataset using pandas.read_csv() and display the top rows using .head() to understand the structure.
3. Check for Missing Values
   Use .isnull().sum() to check for any missing values in the dataset.
4. Create Target Classes
   Convert the continuous charges column into categorical labels like Low, Medium, and High using custom conditions.
5. Drop Original Charges Column
   Remove the charges column from the dataset as it's now represented by the new categorical column.
6. Preprocess the Data
   Convert categorical features into numeric form using get_dummies() and encode the target class using LabelEncoder.
7. Split Data into Train and Test Sets
   Use train_test_split() to divide features and target into training and testing sets (e.g., 80% training and 20% testing).
8. Scale the Features
   Normalize the features using StandardScaler to improve neural network training performance.
9. Train the MLPClassifier Model
   Create an instance of MLPClassifier with desired hidden layers and train it using .fit() on the training data.
10. Make Predictions and Evaluate the Model
    Predict the target for the test set using .predict() and evaluate the model using accuracy_score and classification_report.

PROGRAM

```
# 1. Import required libraries

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, accuracy_score

# 2. Load the dataset

file = pd.read_csv("insurance.csv")

# 3. Check for null values

print("Missing Values:\n", file.isnull().sum())

# 4. Create a new column for classification

def categorize_charge(charge):
    if charge < 10000:
        return "Low"
    elif charge < 20000:
        return "Medium"
    else:
        return "High"

file['charge_category'] = file['charges'].apply(categorize_charge)

# 5. Drop the original 'charges' column

file = file.drop('charges', axis=1)

# 6. Separate features (X) and target (y)

X = file.drop('charge_category', axis=1)
y = file['charge_category']

# 7. One-hot encode the categorical features

X = pd.get_dummies(X, drop_first=True)

# 8. Encode the target labels into integers

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

```
# 9. Split data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42)

# 10. Standardize the features

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 11. Train the MLP Classifier

mlp = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500,
random_state=42)
mlp.fit(X_train, y_train)

# 12. Predict the test data

y_pred = mlp.predict(X_test)

# 13. Evaluate the model

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred,
target_names=label_encoder.classes_)

# 14. Display the results

print("Accuracy:", accuracy)
print("\nClassification Report:\n", report)
```

INPUT

```
age,sex,bmi,children,smoker,region,charges
52,female,44.7,3,no,southwest,11411.685
50,male,30.97,3,no,northwest,10600.5483
18,female,31.92,0,no,northeast,2205.9808
18,female,36.85,0,no,southeast,1629.8335
21,female,25.8,0,no,southwest,2007.945
61,female,29.07,0,yes,northwest,29141.3603
```

OUTPUT

```
Missing Values:
 age         0
sex          0
bmi          0
children     0
smoker       0
region       0
charges      0
dtype: int64
Accuracy: 0.8955223880597015

Classification Report:
               precision    recall  f1-score   support

         High       0.89      0.74      0.81        57
          Low       0.94      0.97      0.96       155
       Medium       0.77      0.86      0.81        56

     accuracy                           0.90       268
    macro avg       0.87      0.85      0.86       268
 weighted avg       0.90      0.90      0.89       268
```

RESULT

The Multilayer Perceptron (MLPClassifier) model was successfully trained on the insurance dataset to classify medical charges into Low, Medium, and High categories. The model achieved an overall accuracy of approximately 89.55%. The classification report shows that the model performed well, especially for the "Low" class with a precision of 0.94 and recall of 0.97. The macro average F1-score was 0.86, indicating balanced performance across all classes. Therefore, the MLPClassifier is effective for this classification task.